COMPUTING LAB - II
ADVANCED SYSTEM SOFTWARE LAB - CSE 5261
MTech - CSE
Week - 3 Basics of Python Programming

Name : Rajat Jain
Reg No : 200913010

Lab Exercises:

1) Write a class called Investment with fields called principal and interest. The constructor should set the values of those fields. There should be a method called value_after that returns the value of the investment after n years. The formula for this is p(1 + i)n, where p is the principal, and i is the interest rate. It should also use the special method __str__ so that printing the object will result in something like below:
Principal - $1000.00, Interest rate - 5.12%

Code

```
class investment:
    def __init__(self,p,i):
        self.principal = p
        self.interest = i
    def value_after(self,n):
        value = self.principal*((1+self.interest)**n)
        return value
    def __str__(self):
        return 'Principal - ${self.principal}, Interest rate -
{self.interest}%'.format(self=self)

ob1 = investment(1000.00,5.12)
print(ob1)
print(f"The value after 5 years is {ob1.value_after(5)}")
```

Output

```
IDLE Shell 3.9.2
File  Edit  Shell  Debug  Options  Window  Help
Python 3.9.2 (tags/v3.9.2:1a79785, Feb 19 2021, 13:44:55) [MSC v.1928 64 bit (AM
D64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\hp\Desktop\Rajat\MTECH_MIT\1stYear\Semister2\AdvancedSystemS
oftware\Lab\Lab3\prog1.py
Principal - $1000.0, Interest rate - 5.12%
The value after 5 years is 8585332.3256832
>>> |
```

2) Write a class called Password_manager. The class should have a list called old_passwords that holds all of the user's past passwords. The last item of the list is the user's current password. There should be a method called get_password that returns the current password and a method called set_password that sets the user's password. The set_password method should only change the password if the attempted password is different from all the user's past passwords. Finally, create a method called is_correct that receives a string and returns a Boolean True or False depending on whether the string is equal to the current password or not.

Code

```
class Password_manager:
    def __init__(self,old) -> None:
        self.old=old

    def getPassword(self):
        return self.old[len(self.old)-1]

    def setPassword(self, newPw):
        if newPw not in self.old:
            self.old.append(newPw)
            print("Password Updated")
        else:
            print("Entered password is an old password")
            newPass=input("Enter the new password: ")
            self.setPassword(newPass)
    def is_correct(self,curPw):
```

```
        if curPw == self.old[len(self.old)-1]:
            return True
        return False

old_passwords=['abc123','abc*123','abc@123','def123','def*123'] #last
password is current password

ob1=Password_manager(old_passwords)

print("Executing get_password function")
print(f"The user's current password is : {ob1.getPassword()}")

print("Executing set_password function")
newPass=input("Enter the new password: ")
ob1.setPassword(newPass)

print("Executing is_correct function")
curPass=input("Enter current password: ")
if ob1.is_correct(curPass)==True:
    print("Password Matched")
else:
    print("Password Not Matched")
```

<u>Output</u>

3) Write a class called Wordplay. It should have a field that holds a list of words. The user of the class should pass the list of words they want to use to the class. There should be the following methods:

• words_with_length(length) — returns a list of all the words of length length
• starts_with(s) — returns a list of all the words that start with s
• ends_with(s) — returns a list of all the words that end with s
• palindromes() — returns a list of all the palindromes in the list
• only(L) — returns a list of the words that contain only those letters in L
• avoids(L) — returns a list of the words that contain none of the letters in L

<u>Code</u>

```
class Wordplay:
    def __init__(self,words):
        self.words = words
    def words_with_length(self,n):
        return_list = []
        for i in self.words:
            if len(i)==n:
                return_list.append(i)
        return return_list
    def starts_with(self,s):
        return_list = []
        for i in self.words:
          if i[0]==str(s):
              return_list.append(i)
        return return_list
    def ends_with(self,s):
        return_list = []
        for i in self.words:
            if i[-1]==str(s):
                return_list.append(i)
        return return_list
    def palindromes(self):
        return_list = []
        for i in self.words:
            if i == i[::-1]:
                return_list.append(i)
        return return_list

    def only(self,L):
        return_list = []
        for i in self.words:
```

```
            match = True
            for j in i:
                if j not in L:
                    match = False
                    break
            if match==True:
                return_list.append(i)
        return return_list
    def avoids(self,L):
        return_list = []
        for i in self.words:
            match = True
            for j in i:
                if j in L:
                    match = False
                    break
            if match==True:
                return_list.append(i)
        return return_list
words = ['apple','ball','cat','dog','owl','bat','tat','pip']
ob1 = Wordplay(words)
print(f"Chosen words are : {words}")
print(f"The words with length 3 are : {ob1.words_with_length(3)}")
print(f"The words that start with 'b' are : {ob1.starts_with('b')}")
print(f"The words that ends with 't' are : {ob1.starts_with('t')}")
print(f"The words which are palindromes are : {ob1.palindromes()}")
print(f" The words which contains only the letters in ['t','a'] are :
{ob1.only(['t','a'])}")
print(f"The words which contains none of the letters in ['p','b','w'] are :
{ob1.avoids(['p','b','w'])}")
```

<u>Output</u>

```
IDLE Shell 3.9.2
File  Edit  Shell  Debug  Options  Window  Help
Python 3.9.2 (tags/v3.9.2:1a79785, Feb 19 2021, 13:44:55) [MSC v.1928 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
=================== RESTART: C:\Users\hp\Downloads\P3 (1).py ===================
Chosen words are : ['apple', 'ball', 'cat', 'dog', 'owl', 'bat', 'tat', 'pip']
The words with length 3 are : ['cat', 'dog', 'owl', 'bat', 'tat', 'pip']
The words that start with 'b' are : ['ball', 'bat']
The words that ends with 't' are : ['tat']
The words which are palindromes are : ['tat', 'pip']
 The words which contains only the letters in ['t','a'] are : ['tat']
The words which contains none of the letters in ['p','b','w'] are : ['cat', 'dog', 'tat']
>>> |
```

4) Write the regular expressions in Python for the following:
i) To replace all occurrences of abc with *.(Eg:- ‘abcdef’ replace to (‘abc’,’*’)
ii) To convert roman numerals into ordinary numbers.
iii) To recognize the strings: “bat,” “bit,” “but,” “hat,” “hit,” or “hut.”
iv) That converts to lowercase every capital letter that is followed by a lowercase letter.

Code

```
import re
text = input("Enter the text")
text = re.sub('abc','*',text)
#print(text)
def romanToInt(rom):
	value = {
		'M': 1000,
		'D': 500,
		'C': 100,
		'L': 50,
		'X': 10,
		'V': 5,
		'I': 1
	}

	# Initialize previous character and answer
	p = 0
	ans = 0

	# Traverse through all characters
	n = len(rom)
	for i in range(n-1, -1, -1):

		# If greater than or equal to previous,
		# add to answer
		if value[rom[i]] >= p:
			ans += value[rom[i]]

		# If smaller than previous
		else:
			ans -= value[rom[i]]

		# Update previous
```

```
            p = value[rom[i]]

        return ans

split_words = text.split(' ')
patt = r'[MDCLXVI]+$'
r = re.compile(patt)
roman_numeral = list(filter(r.match, split_words))
#print(roman_numeral)
for x in roman_numeral:
    text = re.sub(x,str(romanToInt(x)),text)
#print(text)

patt = "[bh][aiu]t"

print("Recognized strings are : ")
print(re.findall(patt,text))

caps = re.findall("[A-Z][a-z]",text)
for x in caps:
    replace = x.lower()
    text = re.sub(x,replace,text)
print("Modified text is : ")
print(text)
```

<u>Output</u>

```
=================== RESTART: C:\Users\hp\Downloads\P4 (1).py ====================
Enter the textabcRajat CSE Cr abc VII CX hit hat
Recognized strings are :
['hit', 'hat']
Modified text is :
*rajat CSE cr * 7 110 hit hat
>>> |
```

5) Write the functions in Python to sort a list of strings by length, rather than alphabetically.

<u>Code</u>

```
str = [ "He is a person", "Apple", "Impossible", "There is ASSW class today"]
print("Input list: ")
print(str)
str.sort(key=len)
```

```
print("Sorted list(based on length): ")
print(str)
```

Output

```
IDLE Shell 3.9.2
File  Edit  Shell  Debug  Options  Window  Help
Python 3.9.2 (tags/v3.9.2:1a79785, Feb 19 2021, 13:44:55) [MSC v.1928 64 bit (AM
D64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\hp\Desktop\Rajat\MTECH_MIT\1stYear\Semister2\AdvancedSystemS
oftware\Lab\Lab3\prog5.py
Input list:
['He is a person', 'Apple', 'Impossible', 'There is ASSW class today']
Sorted list(based on length):
['Apple', 'Impossible', 'He is a person', 'There is ASSW class today']
>>>
```

6) Write a program in Python, that removes single and multiline comments from a given input ‘C’ file.

Code

```
import re
def removeComments(string):
    string = re.sub(re.compile("/\*.*?\*/",re.DOTALL ) ,"" ,string) # remove all
occurrences streamed comments (/*COMMENT */) from string
    string = re.sub(re.compile("//.*?\n" ) ,"" ,string) # remove all occurrence
single-line comments (//COMMENT\n ) from string
    return string

with open('cprogram.c','r') as file:
    text = file.read()
modified_program = removeComments(text)
print("C Program after removing the single line and multi-line comments : ")
print(modified_program)
```

Output

```
IDLE Shell 3.9.2
File  Edit  Shell  Debug  Options  Window  Help
Python 3.9.2 (tags/v3.9.2:1a79785, Feb 19 2021, 13:44:55) [MSC v.1928 64 bit (AM
D64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\hp\Desktop\Rajat\MTECH_MIT\1stYear\Semister2\AdvancedSystemS
oftware\Lab\Lab3\prog6.py
C Program after removing the single line and multi-line comments :
#include<stdio.h>
int main(){

    printf("Hello C");
return 0;
}
>>> |
```

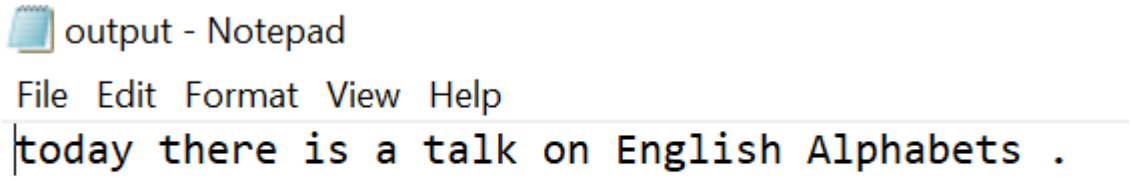
```
IDLE Shell 3.9.2
File  Edit  Shell  Debug  Options  Window  Help
Python 3.9.2 (tags/v3.9.2:1a79785, Feb 19 2021, 13:44:55) [MSC v.1928 64 bit (AM
D64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\hp\Desktop\Rajat\MTECH_MIT\1stYear\Semister2\AdvancedSystemS
oftware\Lab\Lab3\prog6.py
C Program after removing the single line and multi-line comments :
#include<stdio.h>
int main(){
        /* Multi line
    Comment */
    printf("Hello C");
return 0;
}
>>>
```

7) Write a program in Python, that takes a file as input and replaces blank spaces and tabs by single space and writes the output to a file.

Code

```
fin = open("input.txt", "rt")
fout = open("output.txt", "wt")
for line in fin:
        fout.write(' '.join(line.split()))
fin.close()
fout.close()
```

Output



8) Write a program in Python, to match the set of all valid Python identifiers.

Code

```
import re


regex = '^[A-Za-z_][A-Za-z0-9_]*'

def check(string):

    if(re.search(regex, string)):
        print("Valid Identifier")

    else:
        print("Invalid Identifier")

if __name__ == '__main__' :


    string = input("Enter String : ")

    check(string)
```

Output

IDLE Shell 3.9.2

File Edit Shell Debug Options Window Help

```
Python 3.9.2 (tags/v3.9.2:1a79785, Feb 19 2021, 13:44:55) [MSC v.1928 64 bit (AM
D64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\hp\Desktop\Rajat\MTECH_MIT\1stYear\Semister2\AdvancedSystemS
oftware\Lab\Lab3\prog8.py
Enter String : rajat
Valid Identifier
>>> |
```